Freight Rate Prediction

Spotter Machine Learning Engineer Assessment

Hammad Amer | 31 October 2026

1 Summary

This report describes a model that predicts the posted rate of a truckload from its lane, distance, equipment, weight and date. It was trained on 48,000 loads posted between January and October 2025 and used to price 12,000 loads from November and December 2025.

The model has two components. A load model, a 70/30 average of a LightGBM regressor and a log-linear regression, prices each load from its own attributes. A market price level, shared by all loads on a given day, captures how rates move over time: a gradual upward trend and a rise of about 4% over the last four weeks of each quarter.

On three rolling backtests that each forecast two months ahead, the model has a mean absolute error (MAE) of $44 per load (1.9% of the rate), with a worst fold of $51. A median rate-per-mile baseline scores $106, and LightGBM without the market level $77.

Two data issues shaped the approach. About 1.4% of training rates (677 loads) are data-entry errors, two to seven times their expected value, and are excluded from training. The quote_signal column equals the rate per mile in some months, mirrors it in others and carries no information in November and December, so it is not used.

2 Data and data quality

train_test.csv contains 48,000 loads from 1 January to 31 October 2025, each with pickup and delivery city and coordinates, distance, equipment type, weight, posting date, a daily market_index, a quote_signal and the posted rate. validation.csv contains 12,000 loads from 1 November to 31 December 2025 with the same inputs and no rate. Table 1 lists the data-quality issues found and how each was handled.

Table 1. Data-quality issues and their treatment.

Issue	Training	Validation	Treatment
Negative weight	292	145	Sign error; absolute value taken
Missing weight	300	165	Median for the equipment type, plus an indicator
Missing market_index	374	249	Median market_index of the same day
Erroneous rate (1.8x or more from expected)	677	-	Excluded from training; kept in raw test scores
quote_signal unstable over time	All rows	All rows	Not used (Section 3.4)
Cities absent from training (8)	-	1,447 loads	Coordinates used instead of city names
December inputs lack coordinates and market_index	-	31 rows	City coordinate lookup; daily market_index from validation inputs

Erroneous rates were identified by comparing each load's log rate per mile with a ridge regression on lane, equipment, distance, weight, market_index and month, refitted once without the flagged loads so that they

do not distort the estimate. Normal loads lie within about 20% of the estimate and erroneous ones two to seven times away, with no loads in between (Figure 1), so the 1.8x threshold is not sensitive.

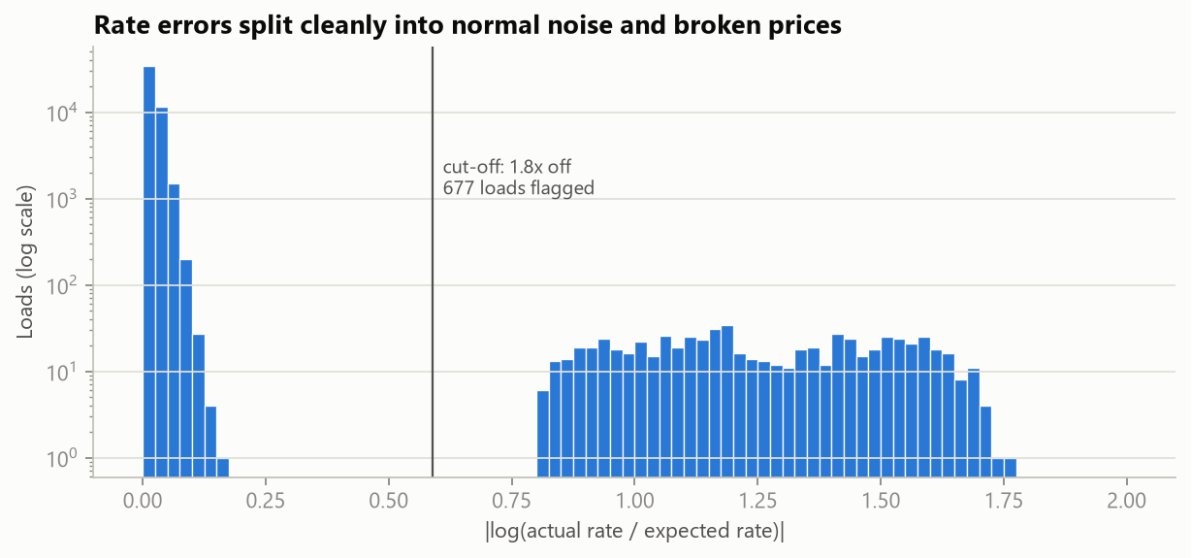


Figure 1. Absolute log ratio of each load's rate to its expected rate. Note the log-scaled count axis.

3 Validation approach and data split

3.1 Splitting by time

The task is a forecast: the model is trained on January to October and applied to November and December. A random split places loads from the same days in both the training and test sets, so the test set contains no unseen market conditions, and it rewards features whose meaning drifts over time. Table 2 compares the two splits for the same LightGBM model without the market level.

Table 2. Test MAE of the same model under a random and a time-based split.

Split	Features	MAE
Random 80/20	with quote_signal	$38.5
Random 80/20	without quote_signal	$51.6
Time: Jan-Aug to Sep-Oct	with quote_signal	$91.8
Time: Jan-Aug to Sep-Oct	without quote_signal	$117.6

The random split understates the error by a factor of two to three, and it favours quote_signal, which is harmful in a true forecast (Section 3.4). All model selection therefore uses time-based folds.

3.2 Rolling time-based folds

Table 3. Backtest folds.

Fold	Training period	Test period	Training loads	Test loads
Jan-Apr to May-Jun	01 Jan to 30 Apr	01 May to 30 Jun	19,110	9,696
Jan-Jun to Jul-Aug	01 Jan to 30 Jun	01 Jul to 31 Aug	28,806	9,671
Jan-Aug to Sep-Oct	01 Jan to 31 Aug	01 Sep to 31 Oct	38,477	9,523

Each fold trains on all data up to a cutoff and tests on the following two months, matching the November to December horizon (Table 3). Erroneous rates are removed from the training data only. Test error is reported on clean loads, which reflects the model's accuracy, and on all loads ('raw'), since the hidden validation labels may contain similar errors. Because the scoring metric is not specified, MAE, RMSE and MAPE are all reported. Models are selected on mean MAE and on worst-fold MAE, to avoid favouring a model that does well in a single period. The selected model is then refitted on all 47,323 clean January to October loads.

3.3 Leakage controls

Lookup tables (daily market_index medians, city coordinates and weight medians) are built from input columns only and never use posted_rate. Validation inputs are used for two purposes only: extending the daily market_index table to November and December, and locating cities that appear only in validation. Within each fold, the models and the market level are fitted on that fold's training period alone. Erroneous rates are flagged once on the full January to October data; flagging each fold's training data separately selects exactly the same loads (275, 400 and 533), because there is a wide gap between normal variation and the threshold.

3.4 quote_signal

quote_signal behaves differently from month to month (Figure 2). In January to March, June and September it is almost exactly the rate per mile (correlation +0.996); in April, May, July and October it is its mirror image (-0.99); in August it is unrelated (-0.02). Its state in November and December cannot be checked against labels, but it can be inferred: in every other labelled month quote_signal is also strongly correlated with distance (absolute correlation of about 0.7), because the rate per mile is. That correlation is +0.02 in November and +0.01 in December, as in August (+0.01). A model trained on the other months and tested on August has an MAE of $80 with quote_signal and $36 without it, so the feature is excluded.

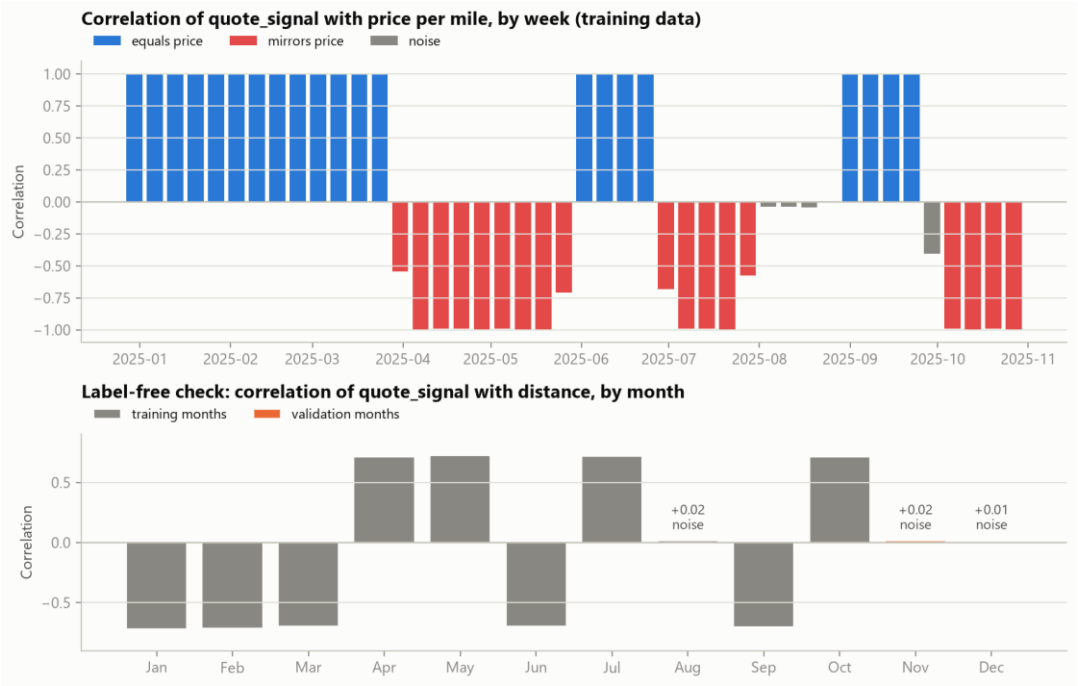


Figure 2. Weekly correlation of quote_signal with the rate per mile (top) and the label-free check by month (bottom).

3.5 Unseen cities

Eight validation cities never appear in training. To test how the models handle new cities, eight training cities (Bakersfield, Hartford, Memphis, New York, San Antonio, Toledo, Tulsa and Washington) were withheld from training and the models were scored on the loads touching them (Table 4). The coordinate-based LightGBM model generalises better than the log-linear model with city indicators, and blending the two costs nothing on these loads.

Table 4. Error on loads involving cities withheld from training.

Model	MAE on loads touching a withheld city
Log-linear (city one-hot)	$70.1
LightGBM (coordinates)	$56.9
Blend	$57.0

4 Model

4.1 Load model

Rates are close to multiplicative in their drivers: shorter trips cost more per mile, Reefer and Flatbed loads cost about 13% and 8% more per mile than Dry Van, and heavier loads cost slightly more. All models therefore predict the logarithm of the rate.

The load model is a 70/30 average of two models. The first is LightGBM (1,500 trees, learning rate 0.03, 31 leaves) using distance and its logarithm, equipment, weight, a missing-weight indicator, market_index, pickup and delivery coordinates, straight-line distance, latitude and longitude differences, and day of week. City

names are excluded so that the model can price unseen cities; tuning the hyperparameters changed backtest MAE by less than $1. The second is a ridge regression on log distance and its square, log weight and log market_index, with indicators for equipment, pickup city, delivery city and day of week. A city it has not seen receives no city effect.

4.2 Market price level

After accounting for load attributes, the rates of all loads on a given day still move together in a way that market_index does not explain. This daily level is estimated jointly with the load model by alternating between the two fits for five iterations. Its final values come from out-of-fold predictions, with days assigned to five groups, so that the load model cannot absorb day-specific effects. The daily level shows two regular patterns (Figure 3): an upward trend of +0.59% per month, and a rise of 4.1% over the last 28 days of each quarter that reverses on the first day of the next quarter. For a future date t, the level is forecast as

$$level(t) = L + b\cdot(t - t_o) + r\cdot q(t)$$

where L is the mean level over the last 14 days of training with the quarter-end effect removed (+2.8% relative to the January to October average), t_o is the middle of that window, b is the trend, r is the quarter-end effect, and q(t) rises linearly from 0 to 1 over the last 28 days of the quarter. The level is modelled separately because tree models cannot extrapolate in time: a date beyond the training range would be treated as the last date seen.

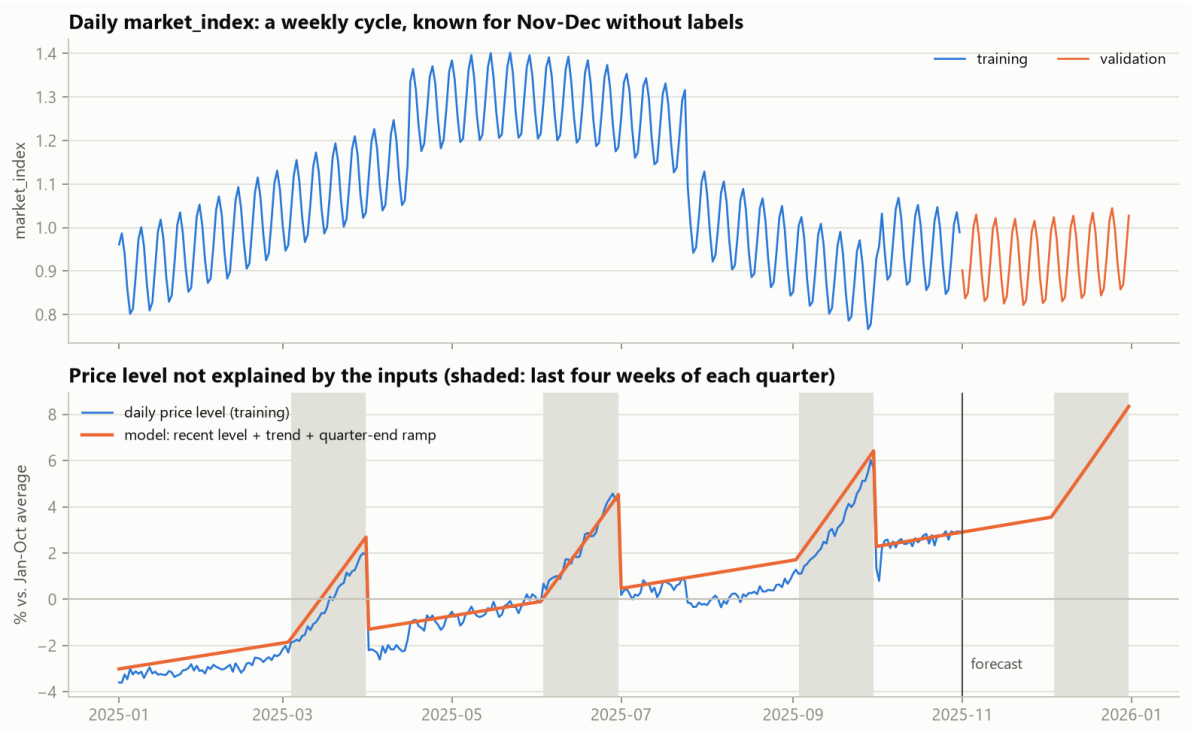


Figure 3. Daily market_index (top) and the estimated daily price level with the fitted and forecast level (bottom).

4.3 Results

Table 5. Backtest results, mean over three folds. MAE, worst fold, RMSE, MAPE and bias are on clean test loads.

Model	MAE	Worst fold	RMSE	MAPE	Bias	Raw MAE	Raw RMSE
Blend + market level	$44	$51	$65	1.89%	+1.2%	$98	$628
LightGBM + market level	$46	$55	$66	1.96%	+1.2%	$100	$629
LightGBM + recent level	$56	$62	$83	2.38%	+0.9%	$110	$631
Log-linear	$75	$115	$104	3.13%	+2.6%	$128	$637
LightGBM	$77	$118	$106	3.24%	+2.9%	$131	$638
Naive $/mile	$106	$148	$143	4.59%	+2.5%	$159	$645

The blend with the market level has the lowest mean and worst-fold MAE (Table 5, Figure 4). Adding the market level is the largest single improvement, reducing LightGBM's error from $77 to $46. Blending with the log-linear model lowers it further to $44 and the worst fold from $55 to $51. Bias is the mean log error on clean loads, where a positive value means under-prediction. Raw RMSE is dominated by erroneous rates, which no model can predict, and is therefore similar across models.

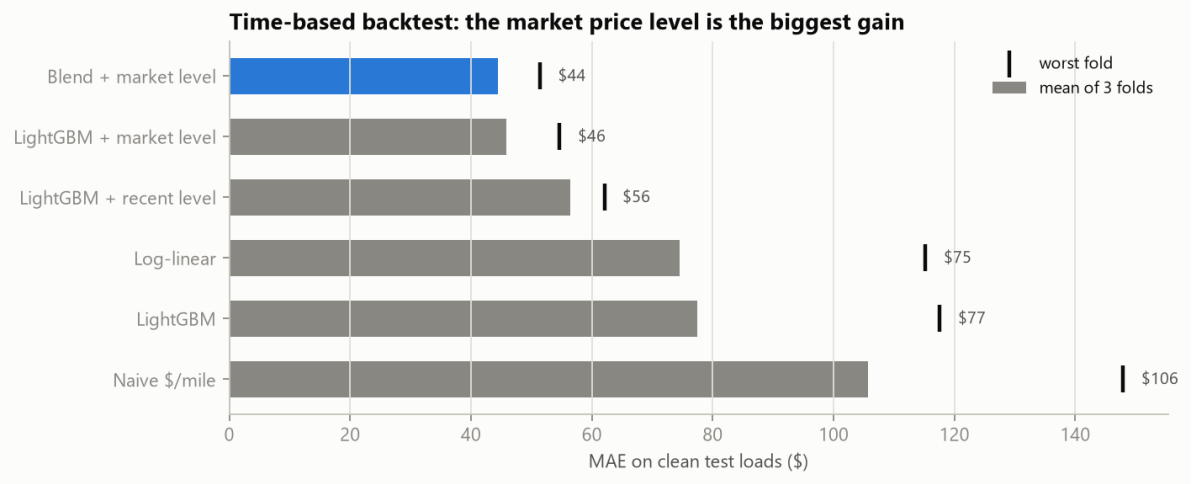


Figure 4. Mean MAE over the three folds (bars) and worst fold (ticks).

5 December prediction chart

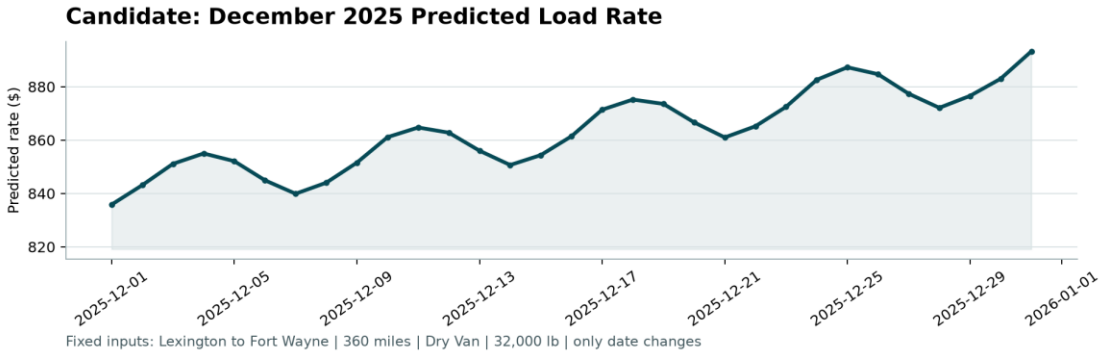


Figure 5. December predictions for the fixed load, as produced by the provided score.py.

Only the date varies (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb). Predictions range from $836 to $893. The weekly oscillation follows market_index, which peaks on Thursdays, and the upward slope reflects the trend and the quarter-end rise that begins in early December. For reference, the 32 earlier loads on this lane, across all equipment types, were priced between $758 and $974. No holiday effect is assumed: the training data contain no December and show no effect around Memorial Day or 4 July.

6 Limitations and next steps

The scoring metric is not specified. The model is optimised for typical error; under raw RMSE, erroneous rates in the validation labels would dominate the score of any model.

The November and December level is an extrapolation. The trend and the quarter-end rise are estimated from three quarters, and the data contain no December, so a year-end holiday effect cannot be learned. In production, the level would be re-estimated weekly from the most recent posted rates.

The selected model under-predicted by 0.7% to 1.8% across the folds, because rates rose slightly faster than a linear trend. No correction was applied, since three folds are too few to estimate one reliably; a damped or piecewise trend is the next candidate.

Natural extensions are prediction intervals (for example, quantile LightGBM) to quantify pricing risk, lane-history features as more data accumulates, and tracing the source of quote_signal.

Appendix A Reproducing the results

With the four provided files in data/, the following commands reproduce every number and figure in this report:

```
python -m pip install -r requirements-dev.txt
python -m src.train
python -m src.diagnostics
python -m src.predict
python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv
python -m pytest
python -m src.report
```

The test suite passes, and a fresh clone of the repository reproduces the submitted predictions exactly.